Cloud-based Research Paper Summarization Tool

1. High-Level Architecture Overview

The system will follow a standard cloud-native microservices or service-oriented architecture (SOA), divided into four primary layers:

- **Presentation Layer (Frontend):** The web interface where users interact with the system.
- **Application Layer (Backend):** The core API that handles business logic, data routing, and API integrations.
- **AI & Processing Layer (LLM Integration):** The engine responsible for document parsing, chunking, and interacting with the Large Language Model.
- **Data & Infrastructure Layer (Storage & Cloud):** The databases, object storage, and hosting environments.

---

2. Component Breakdown

A. Presentation Layer (Frontend)

- **Technology Stack:** React.js, Vue.js, or Next.js (for SEO and fast rendering).
- **Responsibilities:**
    - **Search Interface:** A search bar with filters (e.g., date, author, domain) to query papers.
    - **Results Dashboard:** Displays search results with metadata (title, abstract, authors).
    - **Summarization View:** A dedicated reading panel showing the generated summary (e.g., key contributions, methodology, results) alongside a link to the original paper.
    - **User Management:** Login/Signup, dashboard of previously summarized papers, and saved search history.

B. Application Layer (Backend API)

- **Technology Stack:** Python (FastAPI or Flask) is highly recommended due to its robust ecosystem for NLP and data processing.
- **Core Modules:**
    - **Search Integration Service:** Connects to open-source academic APIs (e.g., arXiv API, Semantic Scholar API, Crossref API, PubMed) to retrieve paper metadata and download links.
    - **Authentication Service:** Manages user sessions using JWT (JSON Web Tokens) or OAuth 2.0 (Google/GitHub login).

- **Job Queue/Asynchronous Processing:** Summarizing large documents takes time. Use a message broker like **RabbitMQ** or **Redis + Celery** to handle LLM requests asynchronously so the frontend doesn't time out.

C. AI & Processing Layer (LLM Engine)

- **Technology Stack:** LangChain or LlamaIndex, Python libraries (PyPDF2, Grobid, unstructured.io).
- **Responsibilities:**
  - **Document Ingestion & Preprocessing:** Downloads the PDF and extracts raw text. Tools like Grobid are excellent for extracting structured data (headings, references) from academic PDFs.
  - **Text Chunking:** Research papers often exceed an LLM's context window. The text must be split into logical, overlapping chunks.
  - **LLM Orchestration:**
    - For shorter papers: A standard summarization prompt.
    - For longer papers: A **Map-Reduce** approach (summarize each chunk, then summarize the summaries) or **Refine** approach.
  - **LLM Provider:** Integration with cloud-based LLM APIs (e.g., Google Gemini, OpenAI GPT-4, or Anthropic Claude).

D. Data & Storage Layer

- **Relational Database (e.g., PostgreSQL or MySQL):** Stores user profiles, authentication data, search history, and saved summary text.
- **Object/Blob Storage (e.g., AWS S3, Google Cloud Storage):** Stores the downloaded raw PDF files temporarily or permanently.
- **Vector Database (Optional but Recommended - e.g., Pinecone, Weaviate, pgvector):** If you plan to add a "Chat with this Paper" feature (RAG), you will need to store vector embeddings of the paper chunks here.
- **Cache (Redis):** Caches frequent search queries or recently generated summaries to reduce API costs and improve load times.

---

3. Cloud Infrastructure & Deployment

To ensure the system is scalable and reliable, it should be deployed using modern DevOps practices.

- **Containerization:** Use **Docker** to containerize the frontend, backend, and background workers.
- **Container Orchestration:** Deploy on managed services like **AWS ECS/EKS**, **Google Kubernetes Engine (GKE)**, or fully managed serverless platforms like **Google Cloud Run**.

- **API Gateway / Load Balancer:** Routes incoming user traffic to the appropriate backend services and handles rate limiting.
- **CI/CD Pipeline:** GitHub Actions or GitLab CI to automate testing and deployment whenever new code is pushed.

---

4. System Data Flow (Step-by-Step)

1. **Search Initiation:** The user enters a topic (e.g., "Transformer models in healthcare") on the React frontend.
2. **Metadata Retrieval:** The request hits the FastAPI backend, which queries the arXiv/Semantic Scholar APIs and returns a list of relevant papers.
3. **Selection & Download:** The user selects a paper to summarize. The backend fetches the PDF from the source repository and stores a copy in the S3 Object Storage.
4. **Preprocessing:** A background worker (Celery) picks up the task, reads the PDF from S3, strips out the references/images, and extracts the core text.
5. **Chunking & LLM Request:** LangChain chunks the text and sends a structured prompt to the LLM API requesting an academic summary (Objectives, Methods, Results, Conclusion).
6. **Storage & Delivery:** The LLM returns the summary. The backend saves this summary to the PostgreSQL database under the user's profile and updates the cache.
7. **Notification:** The backend notifies the frontend (via WebSockets or standard polling) that the summary is ready, and it is displayed to the user.